

Laboratório de Sistemas Operacionais - Trabalho 2

Prof. Sérgio Johann Filho

Objetivos

- Expandir o conhecimento sobre construção e suporte de distribuições do sistema operacional Linux.
- Compreensão das estruturas de dados e interfaces internas do kernel Linux.
- Projeto e implementação de *device drivers*.

Descrição

Este trabalho consiste no projeto implementação de um módulo carregável de kernel (KLM - *kernel loadable module*). Esse módulo terá como objetivo implementar um mecanismo para distribuição de mensagens entre processos utilizando o modelo Publish/Subscribe (acessível em `/dev/pubsub`). O sistema de mensagens poderá ser acessado por meio de uma múltiplas instâncias de uma aplicação que faz uso da API de acesso ao driver. Devem existir múltiplos processos em espaço de usuário, e cada um poderá realizar operações de leitura ou escrita no driver. Dessa forma, o driver será responsável por implementar a funcionalidade de um *broker*, ou seja, manter diferentes tópicos, os processos associados a cada tópico e a distribuição de mensagens.

As operações de escrita deverão enfileirar mensagens em uma estrutura de dados gerenciada no próprio driver (em espaço de kernel), de tal forma que um processo possa consumir posteriormente mensagens enviadas ao driver e a ele endereçadas. Um processo é destino de uma mensagem se estiver escrito em um tópico e uma mensagem for enviada a esse tópico, sendo que um processo poderá estar inscrito ou não em um ou mais tópicos.

Para o desenvolvimento do módulo carregável, deverão ser utilizadas as funções *kmalloc()* e *kfree()* bem como a API de listas como estruturas de dados disponíveis em nível de kernel. É importante que a gerência de memória seja implementada corretamente, evitando o travamento do sistema e vazamentos de memória.

Funcionamento

O módulo deverá ser carregado da forma ilustrada abaixo, sendo necessário informar os seguintes parâmetros ao módulo: o número máximo de mensagens enfileiradas por tópico (para cada processo inscrito em um tópico) e o tamanho máximo de cada mensagem (em bytes). Importante ressaltar que, caso um processo receba mais mensagens do que o número máximo (em cada tópico), as mensagens mais antigas deverão ser descartadas.

```
$ modprobe pubsub_driver 5 250
```

Após a sua carga, o módulo (driver) deverá atender solicitações de processos em nível de usuário, sendo o seu comportamento implementado em uma aplicação. Antes de poder receber mensagens de um tópico, um processo deve inscrever-se em um tópico, associando-se ao mesmo por meio de uma operação de escrita em */dev/pubsub* (o módulo do kernel deve associar o *pid* do processo ao tópico). Um processo poderá realizar múltiplas inscrições em diferentes tópicos.

```
/subscribe topic1
```

Adicionalmente, um processo poderá remover sua inscrição em um tópico:

```
/unsubscribe topic1
```

Um processo pode enviar dados a outros processos por meio de uma operação de escrita. Por exemplo, para enviar uma mensagem a um tópico com o nome "topic1", um outro processo pode realizar uma operação de escrita em */dev/pubsub* com uma mensagem no seguinte formato, a ser enviado como uma string para o módulo:

```
/publish topic1 "tudo bem por ai?"
```

Diversas solicitações desse tipo podem ser enviadas por diferentes processos, sendo que o driver deverá manter uma lista de processos associados a cada tópico, e associado a cada processo dessa lista deverá existir outra lista (fila circular), contendo as últimas mensagens destinadas a cada processo. Isso é necessário para que cada processo possa consumir uma cópia da mensagem enviada ao tópico, no seu tempo. Posteriormente, processos podem solicitar mensagens de um tópico e realizar operações de leitura. Cada operação de leitura efetuada por um processo removerá uma única mensagem da sua fila de mensagens de um tópico. Para realizar a leitura de mensagens destinadas a um tópico, um processo deverá inicialmente realizar uma operação de escrita, seguido de operações de leitura:

```
/fetch topic1
```

Após a seleção do tópico um processo realizará operações de leitura. Caso o conteúdo retornado seja nulo, significa que não existem mais mensagens destinadas ao processo naquele tópico.

Observações adicionais

Na carga do módulo devem ser informados dois parâmetros (número máximo de mensagens enfileiradas por tópico para cada processo e tamanho máximo de mensagens). Caso ocorra o acúmulo de mensagens ou seja enviada uma mensagem maior que o limite de tamanho, deve ser emitido um alerta no log, e a mensagem deve ser descartada. Caso um processo remova sua inscrição de um tópico e existirem mensagens não lidas por ele nesse tópico, o driver deverá emitir um alerta.

Para lançar múltiplos processos pelo terminal do Linux embarcado, pode-se carregar a aplicação e em determinado momento colocá-la em *background* usando as teclas Ctrl+Z. Dessa forma, pode-se carregar mais instâncias da mesma aplicação. Para listar os processos colocados em segundo plano, digite *jobs*. Para resumir a execução de um processo, utilize *fg %n* onde *n* representa o número do *job*.

Apresentação e entrega

Este trabalho deverá ser realizado em grupos de 3 ou 4 integrantes e apresentado no dia 26/09 (apresentação de 5 a 10 minutos). Para a entrega, é esperado que seja enviado pelo Moodle um arquivo *.tar.gz* do projeto com o seu nome e contendo:

1. Código fonte do módulo de kernel desenvolvido;
2. Código fonte da aplicação de usuário desenvolvida para validação do módulo;
3. Scripts para compilação e instalação do módulo e aplicação na distribuição Linux.